

Notatka na kolokwium

Funkcje z 1. semestru & pełny projekt analizy danych (housing)

Python od podstaw • pandas • NumPy • scikit-learn • SimpleImputer,
StandardScaler, OneHotEncoder

Materiał opracowany na podstawie repozytorium **KGN_Programowanie2**: notatnika **funkcje_semestr_1.ipynb** oraz projektu **P2Lab10 (housing)** w wariantach analizy i ML. Notatka tłumaczy każdą funkcję i każdą linijkę kodu projektu od zera, tak aby po jej przerobieniu swobodnie operować omawianymi pojęciami i narzędziami.

Spis treści

CZĘŚĆ I — Funkcje z pierwszego semestru (podstawy Pythona)

1. Funkcje, typy podstawowe i instrukcje warunkowe
2. Pętle: while, for i funkcja range
3. Listy i wyrażenia listowe
4. Krotki, indeksowanie i wycinki
5. Napisy (ciągi znaków)
6. Słowniki
7. Rekurencja
8. Obsługa wyjątków (try/except)
9. Wyrażenia regularne (regex)
10. Pliki i format JSON

CZĘŚĆ II — Projekt analizy danych: California Housing

1. Wczytanie i eksploracja danych
2. Podział na zbiór treningowy i testowy (stratyfikacja)
3. Wizualizacje i korelacje
4. Oddzielenie etykiet (X i y)
5. Czyszczenie i przygotowanie danych — potok (pipeline)
6. Trenowanie i ocena modeli
7. Strojenie hiperparametrów i ocena końcowa

CZĘŚĆ III — Kluczowe narzędzia scikit-learn (dogłębnie)

1. Wspólny interfejs sklearn: fit/transform
2. SimpleImputer — uzupełnianie braków
3. StandardScaler — standaryzacja
4. OneHotEncoder — kodowanie kategorii
5. Pipeline i ColumnTransformer
6. Modele, metryki i walidacja — podsumowanie

CZĘŚĆ I

Funkcje z pierwszego semestru

— podstawy Pythona

1. Funkcje, typy podstawowe i instrukcje warunkowe

Zanim przejdziemy do konkretnych zadań, ustalmy fundament. **Funkcja** to nazwany fragment kodu, który przyjmuje dane wejściowe (argumenty), wykonuje obliczenia i zwraca wynik (słowem kluczowym return). Definiujemy ją słowem **def**. Dzięki funkcjom kod piszemy raz, a używamy wielokrotnie.

Definicja: Składnia: **def nazwa(argumenty):** a pod spodem, wcięty o spację/tabulator, blok instrukcji. Wcięcie (indentacja) w Pythonie nie jest kosmetyką — to ono wyznacza, co należy do funkcji. return kończy działanie funkcji i oddaje wartość.

1.1. Typy podstawowe (proste)

Typ	Znaczenie	Przykład
int	liczba całkowita	5, -3, 0
float	liczba zmiennoprzecinkowa (ułamkowa)	3.14, -0.5
bool	wartość logiczna	True, False
str	napis (ciąg znaków)	'ala', "kot"
None	brak wartości	None

1.2. Funkcja is_even — parzystość, operator modulo, typ bool

Pierwsze zadanie: sprawdzić, czy liczba całkowita jest parzysta. Kluczem jest operator % (modulo) — zwraca resztę z dzielenia. Liczba jest parzysta dokładnie wtedy, gdy reszta z dzielenia przez 2 wynosi 0.

Zadanie 1: sprawdzanie parzystości.

```
# Składnia 'i: int' to PODPOWIEDŹ TYPU (type hint): mówi, że argument ma być int.
# '-> bool' informuje, że funkcja zwraca wartość logiczną. Hints są opcjonalne
# (Python ich nie wymusza), ale czynią kod czytelniejszym.
def is_even(i: int) -> bool:
    return i % 2 == 0      # i % 2 to reszta z dzielenia; == 0 daje True/False

is_even(2)                # 2 % 2 == 0 -> True
```

Wynik:
True

Uwaga: Wyrażenie `i % 2 == 0` samo w sobie jest typu bool, więc nie trzeba pisać `if ...: return True` `else: return False` — wystarczy zwrócić to wyrażenie.

1.3. Funkcja max_of_three — instrukcje warunkowe i szukanie maksimum

Zadanie: zwrócić największą z trzech liczb. Pokazuje ono **instrukcję warunkową if** oraz wzorec „przechowuj dotychczasowego lidera”. Zbieramy liczby do listy i przechodzimy ją pętlą, zapamiętując największy dotąd element:

Zadanie 2: największa z trzech liczb.

```
def max_of_three(a, b, c):
    l = [a, b, c]          # umieść trzy liczby w liście
    counter = l[0]         # załóż, że pierwsza jest największa (kandydat)
    for elem in l:         # przejdź po wszystkich elementach
        if elem > counter:  # jeśli któryś jest większy od kandydata...
            counter = elem  # ...to on staje się nowym kandydatem
    return counter         # po pętli counter trzyma maksimum

max_of_three(-3, 8, 8)    # -> 8
```

Wynik:
8

Uwaga: W notatniku zostawiono zakomentowaną PRÓBĘ z warunkiem ``a > b | c``. To częsty błąd początkującego: `|` to bitowy operator OR, a nie „lub”, i nie znaczy „a większe od b oraz od c”. Poprawnie byłoby ``a > b and a > c``. Wersja z pętlą jest pewniejsza i działa dla dowolnej liczby elementów.

2. Pętle: while, for i funkcja range

Pętla powtarza blok kodu. W Pythonie są dwie: **while** (powtarzaj, dopóki warunek jest prawdziwy) oraz **for** (przejdź po elementach kolekcji albo zakresu).

2.1. count_down — pętla while i wypisywanie

Zadanie: wypisać liczby od n do 0, każdą w nowej linii. Pętla while działa, dopóki warunek jest spełniony; wewnątrz musimy zmieniać zmienną, inaczej pętla nigdy się nie skończy (pętla nieskończona).

Zadanie 3: odliczanie w dół (while).

```
def count_down(n):
    while n != 0:          # powtarzaj, dopóki n jest różne od zera
        print(n)          # wypisz aktualną wartość
        n -= 1            # zmniejsz n o 1 (skrót od n = n - 1)

count_down(5)            # wypisze 5, 4, 3, 2, 1
```

Wynik:
5
4
3
2
1

2.2. count_down_rev — odliczanie w górę i pętla for z range

Tę samą logikę można odwrócić (liczyć w górę). Wersja z while pokazana niżej, a obok wygodniejsza alternatywa z for i funkcją **range**, która generuje kolejne liczby:

Zadanie 3b: odliczanie w górę.

```
# Wersja while:
def cout_down_rev(n):
    start = 1
    while start != n + 1:    # licz od 1 aż do n włącznie
        print(start)
        start += 1

# Równoważna, krótsza wersja z for + range:
def cout_down_rev2(n):
    for elem in range(1, n + 1):    # range(1, n+1) = 1,2,...,n (n+1 jest WYŁĄCZONE)
        print(elem)

cout_down_rev(5)                # 1, 2, 3, 4, 5
```

Wynik:

```
1
2
3
4
5
```

Definicja: range(start, stop, step) tworzy ciąg liczb od start do stop-1 (stop jest WYŁĄCZONY!) z krokiem step. range(5) to 0,1,2,3,4; range(1,6) to 1,2,3,4,5; range(10, 0, -1) liczy w dół.

3. Listy i wyrażenia listowe (list comprehensions)

Definicja: Lista to uporządkowany, ZMIENIALNY (mutable) zbiór elementów w nawiasach kwadratowych: [1, 2, 3]. Elementy mogą być dowolnego typu, indeksujemy je od 0. Najważniejsze metody: append (dodaj na koniec), len (długość).

3.1. sum_list — sumowanie elementów (akumulator)

Zadanie: zsumować elementy listy. Wzorzec **akumulatora**: zaczynamy od 0 i w pętli dodajemy kolejne elementy:

Zadanie 4: suma elementów listy.

```
def sum_list(lst):
    counter = 0          # akumulator startuje od zera
    for elem in lst:     # przejdź po wszystkich elementach listy
        counter += elem  # dodaj element do sumy
    return counter

sum_list([1, 1, 1, 1, 1])    # -> 5
```

Wynik:

```
5
```

Uwaga: Python ma wbudowaną funkcję sum(lst), która robi to samo. Pisanie własnej wersji służy nauce wzorca pętli akumulującej.

3.2. count_positive — filtrowanie i metoda append

Zadanie (wg treści): policzyć elementy dodatnie. Implementacja z notatnika zbiera dodatnie elementy do nowej listy metodą append i ją zwraca:

Zadanie 5: dodatnie elementy (wersja z notatnika).

```
def count_positive(lst):
    new = []          # pusta lista na wyniki
    for elem in lst:
        if elem > 0:  # warunek: tylko liczby dodatnie
            new.append(elem) # dołącz element na koniec listy 'new'
    return new

count_positive([-1, 2, 3, 4, 0, -9]) # -> [2, 3, 4]
```

Wynik:
[2, 3, 4]

Uwaga: Uwaga merytoryczna: treść mówi „zwracająca LICZBĘ elementów dodatnich”, a powyższy kod zwraca LISTĘ tych elementów. Aby zwrócić liczbę, należałoby zwrócić `len(new)` albo zliczać licznikiem. Poprawna wersja: zamień ostatnią linię na ``return len(new)``. Warto znać tę różnicę na kolokwium.

3.3. Wyrażenia listowe (list comprehension)

To zwięzły sposób tworzenia listy w jednej linii. Schemat: [wyrażenie for element in kolekcja if warunek]. Czyta się jak zdanie: „weź wyrażenie dla każdego elementu, dla których warunek jest prawdziwy”.

List comprehension — krótszy zapis pętli budującej listę.

```
# Lista kwadratów liczb 0..4:
[x**2 for x in range(5)]          # -> [0, 1, 4, 9, 16]

# count_positive jako jedna linia (wersja zwracająca liczbę):
def count_positive2(lst):
    return len([e for e in lst if e > 0]) # długość listy dodatnich = ich liczba
```

4. Krotki, indeksowanie i wycinki (slicing)

Definicja: Krotka (tuple) to uporządkowany, ale NIEZMIENIALNY (immutable) zbiór elementów w nawiasach okrągłych: (1, 2, 3). Po utworzeniu nie da się zmienić jej zawartości. Używamy jej dla danych, które nie powinny się zmieniać (np. współrzędne).

4.1. Indeksowanie i wycinki

Elementy numerujemy od 0. Indeksy ujemne liczą od końca (-1 to ostatni). **Wycinek** [start:stop:step] zwraca fragment; szczególnie [::-1] odwraca kolejność:

Indeksowanie i wycinki na krotce.

```
t = (10, 20, 30, 40)
t[0]      # 10 -> pierwszy element
t[-1]     # 40 -> ostatni element
t[1:3]    # (20, 30) -> od indeksu 1 do 3 (3 WYŁĄCZNIE)
t[::-1]   # (40, 30, 20, 10) -> krok -1 odwraca kolejność
```

4.2. reverse_tuple — odwracanie krotki

Zadanie 6: odwrócenie krotki jednym wycinkiem.

```
def reverse_tuple(t):
    return t[::-1]          # wycinek z krokiem -1 = nowa krotka od końca

reverse_tuple((1, 2, 3, 4)) # -> (4, 3, 2, 1)
```


Wynik:
(4, 3, 2, 1)

5. Napisy (ciągi znaków)

Definicja: Napis (str) to niezmienny ciąg znaków. Możemy go indeksować i wycinać jak krotkę, iterować po znakach pętlą for, sprawdzać zawieranie operatorem in, oraz używać metod takich jak split (podziel na słowa), lower/upper (zmiana wielkości).

5.1. count_vowels — zliczanie samogłosek, operator in

Zadanie 7: liczenie samogłosek.

```
def count_vowels(text):  
    vowels = 'a,o,e,u,i,y' # zbiór samogłosek (jako napis)  
    counter = 0  
    for ch in text:  
        if ch in vowels: # 'in' sprawdza, czy znak należy do napisu vowels  
            counter += 1  
    return counter  
  
count_vowels('mateusz') # a, e, u -> 3
```

Wynik:
3

Uwaga: Drobiazg: vowels zawiera też przecinki, ale ponieważ sprawdzamy pojedyncze litery słowa, nie wpływa to na wynik. Czyściej byłoby napisać vowels = 'aoeuiy'.

5.2. is_palindrome — palindrom przez porównanie z odwróceniem

Palindrom czyta się tak samo od przodu i od tyłu. Najprościej: porównać napis z jego odwróceniem (wycinek[::-1]):

Zadanie 8: sprawdzanie palindromu.

```
def is_palindrome(text):  
    return text == text[::-1] # napis równy swojemu odwróceniu?  
  
is_palindrome('anna') # 'anna' == 'anna' -> True
```

Wynik:
True

5.3. word_lengths — split i długości słów

Zadanie łączy metodę split (dzieli napis na listę słów po spacjach) z wyrażeniem listowym i funkcją len:

Zadanie 9: długości kolejnych słów.

```
def word_lengths(sentence):  
    lst = sentence.split() # 'ala ma kota' -> ['ala', 'ma', 'kota']  
    return [len(i) for i in lst] # długość każdego słowa  
  
word_lengths('ala ma kota') # -> [3, 2, 4]
```

Wynik:
[3, 2, 4]

6. Słowniki

Definicja: Słownik (dict) przechowuje pary KLUCZ → WARTOŚĆ w nawiasach klamrowych: {'a': 1, 'b': 2}. Klucze są unikalne; wartość pobieramy przez d[klucz]. Metoda keys() zwraca klucze, values() wartości, items() pary. Sprawdzenie klucz in d mówi, czy klucz istnieje.

6.1. invert_dict — zamiana kluczy z wartościami

Zadanie 10: odwrócenie słownika.

```
def invert_dict(d):
    new = {}
    for key in d.keys():
        new[d[key]] = key
    return new

invert_dict({'a': 1, 'b': 2, 'c': 3}) # -> {1: 'a', 2: 'b', 3: 'c'}
```

Wynik:
{1: 'a', 2: 'b', 3: 'c'}

Uwaga: Działa poprawnie tylko gdy wartości są UNIKALNE — inaczej różne klucze nadpisałyby się po odwróceniu. To założenie podano w treści zadania.

6.2. count_occurrences — zliczanie licznosci (histogram)

Bardzo częsty wzorec: zliczyć, ile razy występuje każdy element. Słownik mapuje element → licznik. Jeśli element już widzieliśmy, zwiększamy licznik; jeśli nie — zakładamy 1:

Zadanie 11: słownik licznosci elementów.

```
def count_occurrences(lst):
    d = {}
    for elem in lst:
        if elem in d:
            d[elem] += 1
        else:
            d[elem] = 1
    return d

count_occurrences([1, 1, 2, 2, 3, 3, 4, 4]) # -> {1: 2, 2: 2, 3: 2, 4: 2}
```

Wynik:
{1: 2, 2: 2, 3: 2, 4: 2}

7. Rekurencja

Definicja: Rekurencja to funkcja, która wywołuje samą siebie dla mniejszego przypadku. Wymaga DWÓCH elementów: (1) warunku bazowego, który zatrzymuje wywołania, oraz (2) kroku rekurencyjnego sprowadzającego problem do prostszego. Bez warunku bazowego dostaniemy nieskończoną rekursję i błąd przepełnienia stosu.

7.1. factorial — silnia rekurencyjnie

Silnia: $n! = n \cdot (n-1) \cdot \dots \cdot 1$, przy czym $0! = 1$. Definicja sama jest rekurencyjna: $n! = n \cdot (n-1)!$.

Zadanie 12: silnia rekurencyjna.

```
def factorial(n):
    if n == 0:                # WARUNEK BAZOWY: 0! = 1, zatrzymuje rekurencję
        return 1
    else:                    # KROK REKURENCYJNY:
        return n * factorial(n - 1)    # n! = n * (n-1)!

factorial(8)                # 8*7*6*5*4*3*2*1 = 40320
```

Wynik:
40320

Uwaga: Prześledź factorial(3): 3*factorial(2) -> 3*(2*factorial(1)) -> 3*(2*(1*factorial(0))) -> 3*(2*(1*1)) = 6. Wywołania „zwijają się” po dotarciu do warunku bazowego.

8. Obsługa wyjątków (try / except)

Definicja: Wyjątek to błąd zgłaszany w trakcie działania programu (np. dzielenie przez zero). Blok try/except pozwala go „złapać” i obsłużyć zamiast przerywać program. W try umieszczamy kod, który MOŻE się nie powieść; w except — reakcję na błąd.

8.1. safe_divide — bezpieczne dzielenie

Zadanie 13: dzielenie odporne na dzielenie przez zero.

```
def safe_divide(a, b):
    try:
        return a / b                # próba dzielenia (może rzucić wyjątek)
    except ZeroDivisionError as e:  # łapiemy KONKRETNĄ wyjątek dzielenia przez 0
        print(f"{e}")              # wypisz komunikat błędu...
        # funkcja nic nie zwraca -> automatycznie zwróci None

safe_divide(6, 0)                  # wypisze: division by zero, zwróci None
```

Wynik:
division by zero

Uwaga: Łapiemy konkretny typ ZeroDivisionError, a nie wszystkie błędy — to dobra praktyka: obsługujemy dokładnie tę sytuację, której się spodziewamy. `as e` daje dostęp do obiektu wyjątku (zawiera komunikat).

9. Wyrażenia regularne (regex)

Definicja: Wyrażenie regularne to wzorzec opisujący zbiór napisów. Moduł re pozwala wyszukiwać takie wzorce. re.findall(wzorzec, tekst) zwraca LISTĘ wszystkich dopasowań. Wzorzec \d oznacza cyfrę, a + znaczy „jeden lub więcej”, więc \d+ pasuje do całych liczb.

9.1. extract_numbers — wyciąganie liczb z tekstu

Zadanie 14: znajdowanie liczb w napisie.

```
import re                                # moduł wyrażeń regularnych

def extract_numbers(text):
    # r'\d+' to 'surowy' napis (raw string): r zapobiega interpretacji \ przez Pythona.
    # \d = cyfra, + = jedna lub więcej -> dopasuj ciągi cyfr (liczby)
    return re.findall(r'\d+', text)

extract_numbers('asdfha23lh3423hl45l33llhl6')    # -> ['23', '3423', '45', '33', '6']
```

Wynik:
['23', '3423', '45', '33', '6']

Uwaga: findall zwraca liczby jako NAPISY (str), nie int. Aby otrzymać liczby całkowite, trzeba je przekształcić: [int(x) for x in re.findall(r'\d+', text)].

10. Pliki i format JSON

Definicja: JSON (JavaScript Object Notation) to tekstowy format zapisu danych, bardzo podobny do słowników/list Pythona. Moduł json zamienia obiekty Pythona na tekst JSON (json.dump — zapis do pliku, json.dumps — do napisu) i odwrotnie (json.load — z pliku, json.loads — z napisu).

10.1. Zapis słownika do pliku JSON

Konstrukcja **with open(...) as plik:** otwiera plik i — co ważne — sam go zamyka po wyjściu z bloku (nawet przy błędzie). Tryb 'w' oznacza zapis (write):

Zapis słownika Pythona do pliku JSON (zadanie 15, część 1).

```
import json

osoba = {"name": "Anna", "surname": "Kowalska", "grades": [4, 5, 3, 5]}

# 'w' = tryb zapisu; with samo zamknie plik po bloku
with open('student.json', 'w') as plik:
    # indent=4 -> ładne wcięcia; ensure_ascii=False -> zachowaj polskie znaki
    json.dump(osoba, plik, indent=4, ensure_ascii=False)
```

10.2. names — odczyt danych z pliku JSON

Odczyt jest analogiczny: tryb domyślny to odczyt ('r'), a json.load zamienia tekst JSON z powrotem na słownik Pythona, z którego pobieramy pola:

Zadanie 15: wczytanie imienia i nazwiska z JSON.

```
import json

def names(filename):
    with open(filename) as plik:
        dane = json.load(plik)          # domyślny tryb 'r' (odczyt)
        # JSON z pliku -> słownik Pythona
        print(f"{dane['name']} {dane['surname']}") # f-string sklejający pola

names('student.json')                  # -> Anna Kowalska
```

Wynik:
Anna Kowalska

Uwaga: f-string (litera f przed cudzysłowem) pozwala wstawiać wartości zmiennych wprost w napis w nawiasach klamrowych {}. To najczytelniejszy sposób formatowania tekstu w Pythonie.

CZĘŚĆ II

Projekt analizy danych

California Housing — krok po kroku

Wprowadzenie do projektu

Projekt rozwiązuje klasyczne zadanie uczenia maszynowego z książki A. Gérona: na podstawie danych o dzielnicach Kalifornii (z amerykańskiego spisu z 1990 r.) przewidzieć **medianę ceny domu** w dzielnicy. To problem **regresji z nadzorem** — cel (`median_house_value`) jest liczbą, a w danych mamy poprawne odpowiedzi do uczenia.

Cały proces przebiega według stałego schematu projektu ML, który warto zapamiętać:

- 1. Wczytanie i eksploracja danych (poznajemy zbiór).
- 2. Podział na zbiór treningowy i testowy (test odkładamy na bok).
- 3. Wizualizacje i analiza korelacji (szukamy zależności).
- 4. Oddzielenie etykiet (X = cechy, y = to, co przewidujemy).
- 5. Czyszczenie i przygotowanie danych w potoku (pipeline).
- 6. Trenowanie kilku modeli i ich porównanie (walidacja krzyżowa).
- 7. Strojenie hiperparametrów najlepszego modelu (Grid/Randomized Search).
- 8. Ostateczna ocena na zbiorze testowym.

1. Wczytanie i eksploracja danych

Najpierw pobieramy plik CSV z danymi. Wykrzyknik ! przed poleceniem oznacza wykonanie komendy systemowej (powłoki) z poziomu notatnika Colab:

Cela [2]: pobranie surowych danych.

```
# Pobranie pliku CSV z danymi (komenda powłoki w Colab)
!wget https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv
```

Wczytujemy dane do ramki pandas. Funkcja `pd.read_csv` parsuje plik CSV i zwraca `DataFrame`:

Cela [3]: wczytanie danych do `DataFrame`.

```
import pandas as pd
import numpy as np

def load_housing_data(housing_path="/content/housing.csv"):
    return pd.read_csv(housing_path) # wczytaj CSV -> DataFrame

housing = load_housing_data() # 'housing' to nasza tabela danych
```

1.1. `housing.info()` — struktura tabeli

Metoda `.info()` wypisuje podsumowanie: liczbę wierszy, nazwy kolumn, liczbę wartości NIEpustych w każdej kolumnie oraz ich typy. To pierwszy krok diagnozy — od razu widać braki danych.

Cela [4]: informacje o tabeli.

```
housing.info()

Wynik:
RangeIndex: 20640 entries (20640 dzielnic)
total_bedrooms    20433 non-null (207 BRAKÓW!) <- reszta kolumn ma 20640
ocean_proximity    20640 non-null object      <- jedyna kolumna tekstowa
dtypes: float64(9), object(1)
```

Uwaga: Najważniejsza obserwacja: `total_bedrooms` ma tylko 20433 wartości zamiast 20640 — brakuje 207. Te braki trzeba będzie uzupełnić (zrobi to `SimpleImputer`). Wszystkie kolumny poza `ocean_proximity` są liczbowe (`float64`).

1.2. `value_counts()` — rozkład kolumny kategorycznej

`ocean_proximity` to kolumna tekstowa (kategoryczna) — opisuje położenie względem oceanu. Metoda `.value_counts()` liczy, ile razy występuje każda kategoria:

Cela [5]: licznosci kategorii.

```
housing["ocean_proximity"].value_counts() # ile dzielnic w każdej kategorii
```

```
Wynik:
<1H OCEAN    9136
INLAND       6551
NEAR OCEAN   2658
NEAR BAY     2290
ISLAND         5
```

Uwaga: To pięć kategorii tekstowych. Modele ML nie rozumieją tekstu — tę kolumnę trzeba zamienić na liczby (zrobi to `OneHotEncoder`, omówiony szczegółowo w Części III).

1.3. `describe()` i histogramy — rozkłady cech liczbowych

`.describe()` podaje statystyki opisowe każdej kolumny liczbowej: licznosc (count), średnią (mean), odchylenie standardowe (std), minimum, kwartyle (25%, 50%=mediana, 75%) i maksimum.

Cela [6]: statystyki opisowe.

```
housing.describe() # statystyki opisowe wszystkich kolumn liczbowych
```

Histogram pokazuje rozkład wartości każdej cechy — ile dzielnic mieści się w danym przedziale. Metoda `.hist()` rysuje histogramy dla wszystkich kolumn liczbowych naraz:

Cela [7]: histogramy wszystkich cech liczbowych.

```
%matplotlib inline # rysuj wykresy wewnątrz notatnika
import matplotlib.pyplot as plt

# bins=50 -> 50 słupków na histogram; figsize -> rozmiar rysunku w calach
housing.hist(bins=50, figsize=(20, 15))
plt.show() # wyświetl wykresy
```

Uwaga: Z histogramów uczymy się, że: (a) `median_income` jest w dziwnej skali (ok. 0–15, bo przeskalowano dochód), (b) `median_house_value` i `housing_median_age` są „ucięte” u góry (sztuczny limit), (c) wiele cech jest „skośnych w prawo” (długi ogon dużych wartości). To ważne, bo modele lepiej działają na cechach o zbliżonej skali — stąd potrzeba standaryzacji.

2. Podział na zbiór treningowy i testowy

Zanim cokolwiek zaczniemy modelować, odkładamy część danych jako zbiór TESTOWY, którego nie wolno nam podglądać do samego końca. Inaczej oszukalibyśmy sami siebie co do jakości modelu (patrz Część I poprzedniej notatki — przeuczenie).

Cela [9]: prosty podział losowy 80/20.

```
from sklearn.model_selection import train_test_split

# test_size=0.2 -> 20% danych na test, 80% na trening
# random_state=42 -> stała losowość, by podział był POWTARZALNY
train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

2.1. Stratyfikacja — reprezentatywny podział

Czysto losowy podział może przez przypadek dać zbiór testowy o innym rozkładzie dochodów niż całość. Ponieważ dochód mocno wpływa na cenę, chcemy, by proporcje grup dochodowych były takie same w treningu i teście. To **stratyfikacja** (podział warstwowy). Najpierw tworzymy pomocniczą kolumnę kategorii dochodu:

Cela [11]/[12]: utworzenie kategorii dochodu income_cat.

```
# Dzielimy dochód przez 1.5 i zaokrąglamy w górę (ceil), by zrobić DYSKRETNE
# kategorie dochodowe (1, 2, 3, ...). To grupuje ciągły dochód w przedziały.
housing["income_cat"] = np.ceil(housing["median_income"] / 1.5)

# Wszystkie kategorie powyżej 5 'spłaszczamy' do 5 (clip = przytnij od góry),
# by nie było rzadkich, licząco małych grup.
housing["income_cat"] = housing["income_cat"].clip(upper=5)
```

Teraz dzielimy dane tak, by w obu częściach proporcje income_cat były zachowane. Służą do tego StratifiedShuffleSplit:

Cela [12]: stratyfikowany podział danych.

```
from sklearn.model_selection import StratifiedShuffleSplit

# n_splits=1 -> jeden podział; test_size=0.2 -> 20% na test
split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

# split.split zwraca INDEKSY wierszy treningowych i testowych,
# dobrane tak, by rozkład income_cat był zachowany w obu zbiorach.
for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train_set = housing.loc[train_index] # wiersze treningowe (po etykietach)
    strat_test_set = housing.loc[test_index]   # wiersze testowe
```

Cela [13]: proporcje kategorii dochodu.

```
# Sprawdzenie: udział każdej kategorii dochodu w całości danych
housing["income_cat"].value_counts() / len(housing)
```

```
Wynik:
3.0  0.350581
2.0  0.318847
4.0  0.176308
5.0  0.114438
1.0  0.039826
```

Kolumna income_cat zrobiła już swoje (posłużyła tylko do podziału), więc ją usuwamy z obu zbiorów, aby wrócić do oryginalnych cech:

Cela [14]/[15]: sprzątanie i kopia zbioru treningowego.

```
# Usunąć pomocniczą kolumnę income_cat z OBU zbiorów
for set_ in (strat_train_set, strat_test_set):
    set_.drop(["income_cat"], axis=1, inplace=True) # axis=1 = kolumna; inplace = w miejscu

housing = strat_train_set.copy() # dalej pracujemy na KOPII zbioru treningowego
```

Uwaga: Dlaczego .copy()? Żeby eksperymenty (dodawanie kolumn, wykresy) nie zmieniały oryginalnego strat_train_set. To zabezpieczenie przed przypadkową modyfikacją danych źródłowych.

3. Wizualizacje i korelacje

Teraz badamy zależności w danych treningowych. Najpierw mapa geograficzna — rysujemy dzielnice według długości i szerokości geograficznej, kolorując je ceną:

Cela [17]: mapa cen domów. $\alpha=0.4$ (przezroczystość) uwidacznia zagęszczenia.

```
housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,
              s=housing["population"]/100,      # rozmiar punktu = liczba ludności / 100
              label="population",
              c="median_house_value",            # KOLOR punktu = cena domu
              cmap=plt.get_cmap("jet"), colorbar=True) # mapa kolorów jet + pasek skali
plt.legend()
```

Uwaga: Wniosek z mapy: najdroższe domy (czerwone) są przy wybrzeżu i w dużych skupiskach ludności — położenie i gęstość zaludnienia mają znaczenie dla ceny.

3.1. Macierz korelacji

Definicja: Korelacja (Pearsona) to liczba z przedziału $[-1, 1]$ mierząca LINIOWĄ zależność dwóch cech. $+1$ = idealna zależność rosnąca, -1 = idealna malejąca, 0 = brak zależności liniowej. Metoda `.corr()` liczy korelacje wszystkich par kolumn.

Cela [18]: korelacja cech z ceną domu.

```
attributes = ["median_house_value", "median_income",
              "total_rooms", "housing_median_age"]
corr_matrix = housing[attributes].corr()      # macierz korelacji wybranych cech

# Jak każda cecha koreluje z CENĄ, posortowane malejąco:
corr_matrix["median_house_value"].sort_values(ascending=False)

Wynik:
median_house_value    1.000000
median_income         0.687  <- NAJSILNIEJSZA
total_rooms           0.135
housing_median_age    0.114
```

Uwaga: `median_income` ma korelację 0.69 z ceną — to zdecydowanie najsilniejszy predyktor. Im wyższy dochód mieszkańców, tym droższe domy. To kluczowa cecha dla modelu.

Macierz wykresów rozrzutu (`scatter_matrix`) rysuje każdą cechę względem każdej innej — pozwala wzrokowo wychycić zależności:

Cela [19]: macierz wykresów rozrzutu.

```
from pandas.plotting import scatter_matrix
attributes = ["median_house_value", "median_income",
              "total_rooms", "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8)) # siatka wykresów par cech
```

Cela [20]: dochód kontra cena (widać też poziome 'linie' od ucięcia ceny).

```
# Powiększenie najsilniejszej zależności: dochód vs cena
housing.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1)
```

3.2. Tworzenie nowych, sensowniejszych cech

Surowe „łączna liczba pokoi w dzielnicy” mało mówi — lepsza jest „liczba pokoi na gospodarstwo”. Tworzymy więc cechy pochodne (feature engineering) jako iloraz istniejących kolumn:

Cela [22]: nowe cechy jako ilorazy kolumn (wektoryzacja pandas).

```
# Cechy względne niosą więcej informacji niż wartości bezwzględne:
housing["rooms_per_household"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["population_per_household"] = housing["population"] / housing["households"]
```

Cele [23]/[24]: korelacja po dodaniu nowych cech (bez kolumny tekstowej).

```
housing_num = housing.drop("ocean_proximity", axis=1) # usuń kolumnę TEKSTOWĄ
corr_matrix = housing_num.corr()                      # korelacje już z nowymi cechami
corr_matrix["median_house_value"].sort_values(ascending=False)
```

Wynik:

median_income	0.687	
rooms_per_household	0.146	<- nowa cecha, lepsza niż total_rooms
bedrooms_per_room	-0.260	<- silna UJEMNA korelacja (nowa, wartościowa cecha)

Uwaga: bedrooms_per_room (udział sypialni w pokojach) ma korelację -0.26 — silniejszą niż surowe total_rooms (0.13). Mniej sypialni na pokój = droższe domy. Dobre cechy pochodne potrafią znacząco poprawić model.

4. Oddzielenie etykiet (X i y)

W uczeniu z nadzorem rozdzielamy CECHY (X — to, na podstawie czego przewidujemy) od ETYKIET (y — to, co przewidujemy). Etykietą jest median_house_value:

Cela [26]: rozdzielenie cech (housing) i etykiet (housing_labels).

```
# X = wszystkie kolumny OPRÓCZ ceny (cechy wejściowe)
housing = strat_train_set.drop("median_house_value", axis=1)

# y = sama kolumna ceny (cel/etykiety); .copy() dla bezpieczeństwa
housing_labels = strat_train_set["median_house_value"].copy()
```

Uwaga: Od tej chwili 'housing' to czyste cechy bez ceny. Etykiety (housing_labels) podamy modelowi osobno w metodzie .fit(X, y). Cena NIE może być częścią X — inaczej model „oszukiwałby”, widząc odpowiedź.

5. Czyszczenie i przygotowanie danych — potok (pipeline)

To serce projektu. Dane wymagają kilku przekształceń: uzupełnienia braków, dodania cech pochodnych, standaryzacji liczb i zakodowania kolumny tekstowej. Zamiast robić to ręcznie, budujemy **potok (Pipeline)** — listę kroków wykonywanych po kolei, automatycznie i powtarzalnie. Szczegóły każdego narzędzia są w Części III; tu pokazujemy, jak składają się w całość.

5.1. Własny transformator dodający cechy (CombinedAttributesAdder)

scikit-learn pozwala pisać własne transformatory. Dziedziczymy po BaseEstimator i TransformerMixin, dzięki czemu nasz obiekt ma standardowe metody fit/transform i pasuje do

potoku. Ten transformator dodaje cechy pochodne, ale operuje na tablicy NumPy (indeksy kolumn):

Cela [28]: własny transformator dodający cechy pochodne.

```
from sklearn.base import BaseEstimator, TransformerMixin

# Indeksy (numery) kolumn w tablicy NumPy, których będziemy używać:
rooms_ix, bedrooms_ix, population_ix, household_ix = 3, 4, 5, 6

class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room=True): # hiperparametr: czy dodać bedrooms_per_room
        self.add_bedrooms_per_room = add_bedrooms_per_room
    def fit(self, X, y=None):
        return self # nic nie 'uczymy' -> zwróć siebie
    def transform(self, X, y=None):
        rooms_per_household = X[:, rooms_ix] / X[:, household_ix] # pokoje / gosp. domowe
        population_per_household = X[:, population_ix] / X[:, household_ix]
        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
            # np.c_ skleja kolumny obok siebie (column stack)
            return np.c_[X, rooms_per_household, population_per_household, bedrooms_per_room]
        return np.c_[X, rooms_per_household, population_per_household]
```

Definicja: Para fit/transform to wzorec scikit-learn. fit UCZY się parametrów na danych (np. mediany, średnie), transform STOSUJE je, przekształcając dane. Tutaj fit nic nie liczy (zwraca self), bo dodanie ilorazu nie wymaga uczenia. TransformerMixin dorzuca za darmo metodę fit_transform (fit + transform w jednym).

5.2. Selektor kolumn (DataFrameSelector)

Pomocniczy transformator wybierający tylko wskazane kolumny z DataFrame i zwracający je jako tablicę NumPy (potrzebną kolejnym krokom potoku):

Cela [29]: selektor wybranych kolumn.

```
class DataFrameSelector(BaseEstimator, TransformerMixin):
    def __init__(self, attribute_names):
        self.attribute_names = attribute_names # nazwy kolumn do wybrania
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        return X[self.attribute_names].values # .values -> z DataFrame na tablicę NumPy
```

5.3. Dwa potoki: liczbowy i kategoriowy + ich połączenie

Liczby i tekst wymagają innego przygotowania, więc budujemy DWA osobne potoki, a potem łączymy je w jeden. Wersja z notatnika ANALIZY używa FeatureUnion i własnego MyLabelBinarizer do kodowania tekstu:

Cela [31]: pełny potok przetwarzania (wersja FeatureUnion z notatnika analizy).

```

from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

num_attribs = ['longitude', 'latitude', 'housing_median_age', 'total_rooms',
               'total_bedrooms', 'population', 'households', 'median_income']
cat_attribs = ["ocean_proximity"]

# POTOK LICZBOWY – kroki wykonywane PO KOLEI na kolumnach liczbowych:
num_pipeline = Pipeline([
    ('selector', DataFrameSelector(num_attribs)), # 1. wybierz kolumny liczbowe
    ('imputer', SimpleImputer(strategy="median")), # 2. uzupełnij braki MEDIANĄ
    ('attrs_adder', CombinedAttributesAdder()), # 3. dodaj cechy pochodne
    ('std_scaler', StandardScaler()), # 4. standaryzuj (średnia 0, std 1)
])

# POTOK KATEGORYCZNY – przetwarza kolumnę tekstową:
cat_pipeline = Pipeline([
    ('selector', DataFrameSelector(cat_attribs)), # 1. wybierz kolumnę tekstową
    ('label_binarizer', MyLabelBinarizer()), # 2. zamień tekst na kolumny 0/1
])

# FeatureUnion uruchamia OBA potoki RÓWNOLEGLE i SKLEJA ich wyniki w jedną tablicę:
full_pipeline = FeatureUnion(transformer_list=[
    ("num_pipeline", num_pipeline),
    ("cat_pipeline", cat_pipeline),
])

```

Uruchomienie potoku — `fit_transform` uczy wszystkie kroki na danych treningowych i od razu je przekształca:

Cele [32]/[33]: wykonanie potoku. Wynik to czysto liczbową tablicę gotową dla modelu.

```

housing_prepared = full_pipeline.fit_transform(housing) # surowe cechy -> gotowa tablica liczb
housing_prepared.shape # (16512, 17) -> 16512 wierszy, 17 kolumn

```

Uwaga: Po przetworzeniu mamy 17 kolumn: 8 oryginalnych liczbowych + 3 cechy pochodne + kategorie `ocean_proximity` zakodowane jako kolumny 0/1. Wszystko liczbowe, bez braków, wystandaryzowane — dokładnie to, czego oczekują modele sklearn.

5.4. Wersja nowocześniejsza: ColumnTransformer + OneHotEncoder

Drugi notatnik (P2Lab10_..._ml) realizuje to samo czyściej, używając wbudowanych narzędzi **ColumnTransformer** (zamiast FeatureUnion + selektory) i **OneHotEncoder** (zamiast własnego binaryzera). To dzisiejszy standard:

Cele [31]/[32] z notatnika ML: ten sam efekt, czytelniej (ColumnTransformer).

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")), # uzupełnij braki medianą
    ('attrs_adder', CombinedAttributesAdder()), # dodaj cechy pochodne
    ('std_scaler', StandardScaler()), # standaryzuj
])
cat_pipeline = Pipeline([
    ('one_hot', OneHotEncoder()), # tekst -> kolumny 0/1
])

# ColumnTransformer SAM kieruje wskazane kolumny do właściwego potoku:
full_pipeline = ColumnTransformer([
    ("num_pipeline", num_pipeline, num_attribs), # te kolumny -> potok liczbowy
    ("cat_pipeline", cat_pipeline, cat_attribs), # te kolumny -> potok kategoriowy
])

housing_prepared = full_pipeline.fit_transform(housing)

```

Uwaga: Różnica: FeatureUnion wymaga ręcznych selektorów kolumn w każdym potoku, a ColumnTransformer przyjmuje listę kolumn jako trzeci element krotki i sam je rozdziela. Efekt końcowy (gotowa tablica liczb) jest taki sam.

6. Trenowanie i ocena modeli

Mając przygotowane dane, trenujemy kolejno trzy modele regresji i porównujemy ich błąd. Miarą jest **RMSE** (pierwiastek z błędu średniokwadratowego) — średnia pomyłka w dolarach.

6.1. Regresja liniowa

Cela [36]: trening regresji liniowej.

```

from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression() # utwórz model
lin_reg.fit(housing_prepared, housing_labels) # UCZENIE: dopasuj do danych (X, y)

```

Cela [37]: błąd RMSE regresji liniowej.

```

from sklearn.metrics import mean_squared_error

housing_predictions = lin_reg.predict(housing_prepared) # przewidywania na treningu
lin_mse = mean_squared_error(housing_labels, housing_predictions) # błąd średniokwadratowy
lin_rmse = np.sqrt(lin_mse) # RMSE = pierwiastek z MSE
lin_rmse

```

Wynik:
68628.0 (średnia pomyłka ok. 68 600 \$)

Uwaga: RMSE $\approx$ 68 600 \$ przy cenach w przedziale 120–265 tys. \$ to dużo — model jest NIEDOU CZONY (underfitting). Regresja liniowa jest zbyt prosta, by uchwycić zależności w danych.

6.2. Walidacja krzyżowa (cross-validation)

Definicja: Walidacja krzyżowa k-krotna (k-fold) dzieli dane treningowe na k części. Model trenuje się k razy, za każdym razem ucząc na k-1 częściach i testując na jednej pominiętej. Otrzymujemy k wyników, których średnia jest rzetelną oceną modelu — bez podglądania prawdziwego zbioru testowego.

Cele [38]/[39]: 10-krotna walidacja krzyżowa regresji liniowej.

```
def display_scores(scores):
    print("Scores:", scores)          # wyniki w każdej z k iteracji
    print("Mean:", scores.mean())     # średni wynik
    print("Standard deviation:", scores.std()) # rozrzut (stabilność)

from sklearn.model_selection import cross_val_score

# scoring='neg_mean_squared_error': sklearn maksymalizuje wynik, więc używa
# UJEMNEGO MSE (większy = lepszy). cv=10 -> 10-krotna walidacja.
lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
                             scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-lin_scores) # minus, by wrócić do dodatniego MSE, potem pierwiastek
display_scores(lin_rmse_scores)
```

Wynik:

Mean: ~69050 Std: ~2730

Uwaga: Sztuczka ze znakiem minus: scoring zwraca UJEMNE MSE (bo sklearn zawsze „maksymalizuje” wynik). Dlatego piszemy `np.sqrt(-lin_scores)`, by uzyskać zwykłe, dodatnie RMSE.

6.3. Drzewo decyzyjne — pułapka przeuczenia

Cele [41]/[42]: trening drzewa i jego błąd na TRENINGU.

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor()
tree_reg.fit(housing_prepared, housing_labels)

tree_predictions = tree_reg.predict(housing_prepared)
tree_rmse = np.sqrt(mean_squared_error(housing_labels, tree_predictions))
print(f"Decision Tree Training RMSE: {tree_rmse:.2f}")
```

Wynik:

Decision Tree Training RMSE: 0.00

Uwaga: RMSE = 0 na danych treningowych wygląda na ideał, ale to ALARM: drzewo nauczyło się danych NA PAMIĘĆ (klasyczne przeuczenie). Prawdziwą jakość pokaże dopiero walidacja krzyżowa.

Cele [43]/[46]: walidacja krzyżowa ujawnia przeuczenie.

```
tree_scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                               scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-tree_scores)
display_scores(tree_rmse_scores) # Decision Tree model is overfitting
```

Wynik:

Mean: ~71400 (gorzej niż regresja liniowa! mimo zerowego błędu na treningu)

Uwaga: Po walidacji drzewo wypada ~71 400 \$ — GORZEJ niż prosta regresja liniowa. To dowód przeuczenia: model świetny na danych, które widział, słaby na nowych. Walidacja krzyżowa jest jedynym sposobem, by to wykryć bez ruszania zbioru testowego.

6.4. Las losowy (Random Forest) — najlepszy z trzech

Definicja: Las losowy to ZESPÓŁ (ensemble) wielu drzew decyzyjnych, z których każde uczy się na losowym podzbiorze danych i cech. Końcowa predykcja to ŚREDNIA przewidywań wszystkich drzew. Uśrednianie redukuje przeuczenie pojedynczych drzew i daje stabilniejszy, dokładniejszy model.

Cela [48]: trening lasu losowego i błąd na treningu.

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = RandomForestRegressor()
forest_reg.fit(housing_prepared, housing_labels)

housing_predictions = forest_reg.predict(housing_prepared)
forest_rmse = np.sqrt(mean_squared_error(housing_labels, housing_predictions))
forest_rmse
```

Wynik:
~18600 (niski błąd treningowy)

Cele [49]/[50]: walidacja krzyżowa lasu losowego.

```
scores = cross_val_score(forest_reg, housing_prepared, housing_labels,
                          scoring="neg_mean_squared_error", cv=10)
forest_rmse_scores = np.sqrt(-scores)
display_scores(forest_rmse_scores)
```

Wynik:
Mean: ~50300 Std: ~2000

Uwaga: Las losowy: ~50 300 \$ na walidacji — wyraźnie najlepszy z trzech modeli. Wciąż jest różnica między błędem treningowym (~18 600) a walidacyjnym (~50 300), więc model nadal trochę się przeucza, ale to najlepszy kandydat do dalszego strojenia.

Model	RMSE (walidacja krzyżowa)	Wniosek
Regresja liniowa	~69 050 \$	niedouczony (za prosty)
Drzewo decyzyjne	~71 400 \$	przeuczony (0 na treningu, źle na walidacji)
Las losowy	~50 300 \$	najlepszy — wybieramy do strojenia

7. Strojenie hiperparametrów i ocena końcowa

Definicja: Hiperparametry to ustawienia modelu, których NIE uczy on sam (np. liczba drzew w lesie). Dobieramy je z zewnątrz. GridSearchCV automatycznie sprawdza wszystkie kombinacje podanych wartości, każdą oceniając walidacją krzyżową, i wybiera najlepszą.

7.1. Grid Search — przeszukiwanie siatki

Cela [51]: definicja i uruchomienie Grid Search.

```

from sklearn.model_selection import GridSearchCV

param_grid = [
    # 1. zestaw: 3 wartości n_estimators × 4 wartości max_features = 12 kombinacji
    {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},
    # 2. zestaw: z bootstrap=False -> 2 × 3 = 6 kombinacji
    {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]},
]

forest_reg = RandomForestRegressor(random_state=42)
# cv=5 -> każda z 12+6=18 kombinacji testowana 5 razy = 90 treningów
grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(housing_prepared, housing_labels) # uruchom przeszukiwanie

```

Cele [52]/[53]: odczyt najlepszych ustawień.

```

grid_search.best_params_      # najlepsza znaleziona kombinacja hiperparametrów
grid_search.best_estimator_   # gotowy, najlepszy model (już wytrenowany)

```

Wynik:
 {'max_features': 6, 'n_estimators': 30} (przykładowo)

7.2. Ostateczna ocena na zbiorze testowym

Dopiero TERAŻ, na samym końcu, sięgamy po odłożony zbiór testowy — by uczciwie oszacować, jak model poradzi sobie z nowymi danymi. Kluczowe: na teście używamy **transform** (NIE **fit_transform**!) tym samym potokiem, co na treningu:

Cele [55]–[60]: przygotowanie testu i ostateczna ocena.

```

best_model = grid_search.best_estimator_      # najlepszy model z Grid Search

X_test = strat_test_set.drop("median_house_value", axis=1) # cechy testowe
y_test = strat_test_set["median_house_value"].copy()        # prawdziwe ceny testowe

# UWAGA: transform, NIE fit_transform! Potok został już nauczony na treningu.
# Używamy mediany i średnich Z TRENINGU, by nie 'podejrzeć' testu.
X_test_prepared = full_pipeline.transform(X_test)

test_pred = best_model.predict(X_test_prepared) # przewidywania na teście
final_rmse = np.sqrt(mean_squared_error(y_test, test_pred)) # ostateczny błąd
final_rmse

```

Wynik:
 ~47700 (finalny RMSE)

Uwaga: To NAJWAŻNIEJSZA zasada całego projektu: zbiór testowy przetwarzamy metodą **transform** z potoku nauczonego na TRENINGU. Gdybyśmy wywołali **fit_transform** na teście, potok policzyłby mediany/średnie z danych testowych — to byłby „przeciek danych” (data leakage), a wynik byłby zawyżony i nieuczciwy.

7.3. Randomized Search — losowe przeszukiwanie

Gdy kombinacji jest bardzo dużo, zamiast sprawdzać wszystkie (Grid Search), losujemy ich ograniczoną liczbę. **RandomizedSearchCV** próbuje wartości z podanych ROZKŁADÓW — często znajduje dobre ustawienia znacznie szybciej:

Cele [63]/[64]: Randomized Search jako szybsza alternatywa Grid Search.


```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint # rozkład losowych liczb całkowitych

forest_reg = RandomForestRegressor(random_state=42)
final_pipeline = Pipeline([('regressor', forest_reg)])

param_distributions = {
    # losuj liczbę drzew z przedziału [3, 200) i liczbę cech z [2, 20)
    'regressor__n_estimators': randint(low=3, high=200),
    'regressor__max_features': randint(low=2, high=20),
}

# n_iter=10 -> wylosuj i przetestuj tylko 10 kombinacji (zamiast wszystkich)
rnd_search = RandomizedSearchCV(final_pipeline, param_distributions=param_distributions,
                                n_iter=10, cv=3,
                                scoring='neg_root_mean_squared_error', random_state=42)
rnd_search.fit(housing_prepared, housing_labels)
rnd_search.best_params_ # najlepsza wylosowana kombinacja

```

Uwaga: Zapis z podwójnym podkreśleniem 'regressor__n_estimators' to sposób adresowania parametru KROKU potoku: nazwa_kroku + '__' + nazwa_parametru. Dzięki temu search wie, którego elementu potoku dotyczy strojony hiperparametr.

CZĘŚĆ III

Kluczowe narzędzia scikit-learn

SimpleImputer · StandardScaler ·

OneHotEncoder · Pipeline

1. Wspólny interfejs scikit-learn: estymatory, transformatory, fit/transform

Cała biblioteka scikit-learn opiera się na kilku spójnych pojęciach. Zrozumienie ich raz sprawia, że każdy nowy obiekt (imputer, skaler, encoder, model) używa się tak samo.

Definicja: ESTYMATOR — dowolny obiekt, który UCZY się czegoś z danych metodą `fit(X[, y])`. Nauczone wartości zapisuje w atrybutach zakończonych podkreśleniem, np. `mean_`, `statistics_`, `categories_`.

Definicja: TRANSFORMATOR — estymator, który dodatkowo PRZEKSZTAŁCA dane metodą `transform(X)` (np. uzupełnia braki, skaluje, koduje). Ma też skrót `fit_transform(X) = fit + transform`.

Definicja: PREDYKTOR (model) — estymator, który po nauczaniu PRZEWIDUJE metodą `predict(X)` (np. `LinearRegression`, `RandomForestRegressor`).

1.1. Złota zasada: fit na treningu, transform na teście

To najważniejsza reguła higieny danych. Parametry przekształceń (mediany, średnie, listy kategorii) wyliczamy WYŁĄCZNIE z danych treningowych. Zbiór testowy tylko przekształcamy tymi samymi parametrami. Naruszenie tej zasady to „przeciek danych” (data leakage) — model widzi informacje z testu i jego ocena staje się zawyżona, nieuczciwa.

Schemat poprawnego użycia każdego transformatora w sklearn.

```
transformator.fit(X_train)          # NAUKA parametrów tylko na treningu
X_train_t = transformator.transform(X_train) # przekształć trening
X_test_t = transformator.transform(X_test)  # przekształć test TYMI SAMYMI parametrami

# Skrót dla treningu (fit + transform naraz):
X_train_t = transformator.fit_transform(X_train)
# Dla testu NIGDY fit_transform – tylko transform!
```

2. SimpleImputer — uzupełnianie braków danych

`SimpleImputer` zastępuje brakujące wartości (NaN) wartością wyliczoną z kolumny. W projekcie housing rozwiązuje problem 207 braków w `total_bedrooms`. Modele sklearn nie potrafią uczyć się z brakami — trzeba je uzupełnić albo usunąć; imputacja zwykle jest lepsza niż usuwanie wierszy.

2.1. Strategie uzupełniania

strategy	Czym wypełnia braki	Kiedy używać
'median'	medianą kolumny	dane liczbowe; ODPORNA na wartości odstające (użyta w housing)
'mean'	średnią kolumny	dane liczbowe o rozkładzie zbliżonym do symetrycznego
'most_frequent'	najczęstszą wartość	dane katégoryczne lub liczbowe
'constant'	stałą (parametr <code>fill_value</code>)	gdy chcemy własną wartość zastępczą

Uwaga: W projekcie wybrano medianę, bo dane o nieruchomościach bywają mocno skośne (kilka ogromnych dzielnic). Mediana nie daje się zaburzyć takim wartościami odstającym, w przeciwieństwie do średniej.

2.2. Jak działa krok po kroku

SimpleImputer: fit liczy mediany, transform je wstawia.

```
import numpy as np
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median") # 1. utwórz imputer (strategia: mediana)

# 2. fit -> policz medianę KAŻDEJ kolumny i zapamiętaj w atrybucie statistics_
imputer.fit(housing_num) # housing_num = tylko kolumny liczbowe
print(imputer.statistics_) # wyliczone mediany wszystkich kolumn

# 3. transform -> wstaw zapamiętane mediany w miejsca NaN; zwraca tablicę NumPy
X = imputer.transform(housing_num)
```

Uwaga: `imputer.fit` liczy mediany na DANYCH TRENINGOWYCH i przechowuje je w `statistics_`. Na zbiorze testowym wywołujemy tylko `transform` — braki w teście wypełni medianą z TRENINGU. To gwarantuje brak przecieku danych.

W projekcie housing `SimpleImputer` jest drugim krokiem potoku liczbowego — działa automatycznie na każdej kolumnie liczbowej, zanim dane trafią do skalera i modelu.

3. StandardScaler — standaryzacja cech

`StandardScaler` sprowadza każdą cechę do tej samej skali: **średnia 0, odchylenie standardowe 1**. To kluczowe, gdy cechy mają różne jednostki/rzędy wielkości (np. dochód w dziesiątkach tysięcy vs. wiek w dziesiątkach).

3.1. Wzór i intuicja

$$z = (x - \mu) / \sigma \quad (\mu = \text{średnia cechy}, \sigma = \text{odchylenie standardowe})$$

Od każdej wartości odejmujemy średnią cechy (centrowanie wokół zera), a wynik dzielimy przez odchylenie standardowe (ujednolicenie rozrzutu). Po tej operacji wszystkie cechy są „równe szansami” — żadna nie dominuje tylko dlatego, że ma większe liczby.

3.2. Po co standaryzować

- Algorytmy liczące ODLEGŁOŚCI (K-Means, KNN) bez skalowania kierują się głównie cechą o największych wartościach.
- Metody gradientowe (regresja logistyczna, sieci, SGD) zbiegają SZYBCIEJ i stabilniej na danych wystandaryzowanych.
- Regularyzacja (Ridge/Lasso) karze wagi sprawiedliwie tylko wtedy, gdy cechy są w tej samej skali.

3.3. Działanie krok po kroku (przykład z wykładu)

StandardScaler: fit zapamiętuje `mean_` i `var_`, transform standaryzuje.

```
import numpy as np
from sklearn.preprocessing import StandardScaler

data = np.array([[160], [170], [180], [190]]) # np. wzrost w cm

scaler = StandardScaler()
scaler.fit(data) # fit: policz średnią i wariancję kolumny
print(scaler.mean_[0]) # 175.0 -> zapamiętana średnia
print(scaler.var_[0]) # 125.0 -> zapamiętana wariancja

scaled = scaler.transform(data) # transform: z = (x - 175) / sqrt(125)
print(scaled.ravel()) # [-1.34, -0.45, 0.45, 1.34] -> średnia 0, std 1
```

```
Wynik:
175.0
125.0
[-1.34164079 -0.4472136  0.4472136  1.34164079]
```

Uwaga: Standaryzacja NIE zmienia kształtu rozkładu (nie czyni go „dzwonowym”) — jedynie przesuwa go do średniej 0 i skaluje do odchylenia 1. Wartości odstające pozostają odstające, choć w nowej skali.

Uwaga: StandardScaler vs MinMaxScaler: Standard centruje wokół 0 (zakres nieograniczony), MinMax ściska liniowo do przedziału [0, 1] wzorem $(x - \min) / (\max - \min)$. W projekcie housing użyto StandardScaler jako ostatniego kroku potoku liczbowego.

4. OneHotEncoder — kodowanie zmiennych kategoriycznych

Modele ML operują na liczbach, a kolumna `ocean_proximity` zawiera tekst (<1H OCEAN, INLAND, ISLAND, NEAR BAY, NEAR OCEAN). OneHotEncoder zamienia każdą kategorię na osobną kolumnę 0/1 (tzw. kodowanie „gorącojedynkowe”).

4.1. Na czym polega kodowanie one-hot

Dla k kategorii powstaje k kolumn. W danym wierszu kolumna odpowiadająca jego kategorii ma 1, a wszystkie pozostałe 0 — stąd „one hot” (jedna gorąca jedynka). Przykład:

<code>ocean_proximity</code>	INLAND	ISLAND	NEAR BAY	NEAR OCEAN	<1H OCEAN
INLAND	1	0	0	0	0
NEAR BAY	0	0	1	0	0
<1H OCEAN	0	0	0	0	1

4.2. Dlaczego nie zwykłe liczby 0,1,2,3,4

Kuszące jest ponumerowanie kategorii (INLAND=0, ISLAND=1, ...). To jednak BŁĄD: model uznałby wtedy, że istnieje porządek i odległości — np. że ISLAND (1) jest „bliżej” INLAND (0) niż NEAR OCEAN (4), albo że NEAR OCEAN jest „4 razy większe”. Kategorie nieuporządkowane nie mają takich relacji. One-hot temu zapobiega: każda kategoria jest niezależną, równorzędną kolumną.

4.3. Działanie krok po kroku

OneHotEncoder: fit poznaje kategorie, transform tworzy kolumny 0/1.

```

from sklearn.preprocessing import OneHotEncoder
import numpy as np

cat = np.array([["INLAND"], ["NEAR BAY"], ["<1H OCEAN"], ["INLAND"]])

encoder = OneHotEncoder()
# fit -> wykryj i zapamiętaj listę unikalnych kategorii (atrybut categories_)
# transform -> zamień każdy wiersz na wektor 0/1
onehot = encoder.fit_transform(cat)

print(encoder.categories_)      # [array(['<1H OCEAN', 'INLAND', 'NEAR BAY'])]
print(onehot.toarray())        # macierz 0/1 (patrz niżej)

```

```

Wynik:
[array(['<1H OCEAN', 'INLAND', 'NEAR BAY'], dtype=object)]
[[0.  1.  0.]
 [0.  0.  1.]
 [1.  0.  0.]
 [0.  1.  0.]]

```

Uwaga: OneHotEncoder domyślnie zwraca MACIERZ RZADKĄ (sparse) — oszczędną strukturę przechowującą tylko jedynki (bo zer jest mnóstwo). Metoda `.toarray()` zamienia ją na zwykłą, gęstą tablicę NumPy. W potoku sklearn radzi sobie z formatem rzadkim sam.

Uwaga: Gdy w zbiorze testowym pojawi się kategoria niewidziana podczas fit, domyślnie wystąpi błąd. Można to obejść parametrem `handle_unknown='ignore'`, który zakoduje nieznaną kategorię jako same zera.

4.4. Alternatywy spotykane w repozytorium

W starszym notatniku analizy zamiast OneHotEncoder użyto LabelBinarizer opakowanego we własną klasę MyLabelBinarizer. Efekt jest podobny (kolumny 0/1), ale LabelBinarizer powstał z myślą o ETYKIETACH (y), nie cechach (X), i jego API gorzej pasuje do potoków — dlatego nowszy notatnik słusznie przechodzi na OneHotEncoder. Istnieje też LabelEncoder, który nadaje kategoriom liczby 0,1,2,... — ale jego do cech nieuporządkowanych NIE należy używać z powodów z punktu 4.2.

5. Pipeline i ColumnTransformer — łączenie kroków

5.1. Pipeline — łańcuch przekształceń

Definicja: Pipeline łączy kilka transformatorów (i opcjonalnie model na końcu) w jeden obiekt. Wywołanie `fit/transform` na potoku uruchamia kroki PO KOLEI: wyjście jednego jest wejściem następnego. Dzięki temu cały preprocessing to jedno wywołanie, identyczne na treningu i teście — koniec z ręcznym powtarzaniem operacji.

Pipeline — trzy przekształcenia w jednym łańcuchu.

```

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Lista par (nazwa_kroku, obiekt). Kroki wykonują się w podanej kolejności:
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")), # 1. uzupełnij braki medianą
    ('attrs_adder', CombinedAttributesAdder()), # 2. dodaj cechy pochodne
    ('std_scaler', StandardScaler()), # 3. standaryzuj wynik
])

# fit_transform: każdy krok uczy się i przekształca, wynik wędruje dalej
X_num = num_pipeline.fit_transform(housing_num)

```

Uwaga: Nazwy kroków ('imputer', 'std_scaler'...) są ważne: pozwalają adresować parametry przy strojeniu, np. 'std_scaler__with_mean'. Wszystkie kroki POZA ostatnim muszą być transformatorami; ostatni może być modelem (wtedy potok ma też metodę predict).

5.2. ColumnTransformer — różne kolumny, różne potoki

Definicja: ColumnTransformer pozwala zastosować RÓŻNE przekształcenia do RÓŻNYCH kolumn w jednym kroku: kolumny liczbowe przepuszcza przez potok liczbowy, a tekstowe przez kategoriowy, po czym SKLEJA wyniki w jedną tablicę. To dziś standardowy sposób przygotowania danych mieszanych (liczby + tekst).

ColumnTransformer kieruje kolumny do właściwych przekształceń i scala wyniki.

```

from sklearn.compose import ColumnTransformer

num_attribs = ['longitude', 'latitude', 'housing_median_age', 'total_rooms',
               'total_bedrooms', 'population', 'households', 'median_income']
cat_attribs = ["ocean_proximity"]

# Każda krotka: (nazwa, transformator, lista_kolumn)
full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs), # te kolumny -> potok liczbowy
    ("cat", OneHotEncoder(), cat_attribs), # ta kolumna -> kodowanie one-hot
])

housing_prepared = full_pipeline.fit_transform(housing) # jedno wywołanie = całe przygotowanie

```

Narzędzie	Rola
Pipeline	łączy kroki SZEREGOWO (jeden po drugim, na tych samych danych)
ColumnTransformer	kieruje RÓŻNE kolumny do różnych potoków i scala wyniki obok siebie
FeatureUnion	uruchamia kilka potoków RÓWNOLEGLE na tych samych danych i scala wyniki (starszy odpowiednik)

6. Modele, metryki i walidacja — szybkie podsumowanie API

6.1. Modele regresji użyte w projekcie

Model	Zasada działania	Wynik w projekcie
LinearRegression	dopasowuje prostą/hyperpłaszczyznę (minimum sumy kwadratów)	niedouczony (~69k)

Model	Zasada działania	Wynik w projekcie
DecisionTreeRegressor	drzewo pytań tak/nie dzielące dane na obszary	przeuczony (~71k)
RandomForestRegressor	średnia z wielu losowych drzew (ensemble)	najlepszy (~50k)

6.2. Metryki i walidacja

Najczęstsze metryki i walidacja krzyżowa w projekcie regresji.

```
from sklearn.metrics import mean_squared_error
import numpy as np

mse = mean_squared_error(y_true, y_pred) # średni kwadrat błędu
rmse = np.sqrt(mse) # RMSE = pierwiastek (w jednostce ceny, czytelny)

from sklearn.model_selection import cross_val_score
# 10-krotna walidacja; minus, bo scoring zwraca UJEMNE MSE
scores = cross_val_score(model, X, y, scoring="neg_mean_squared_error", cv=10)
rmse_scores = np.sqrt(-scores) # 10 wartości RMSE -> średnia = ocena modelu
```

6.3. Najważniejsze metody każdego obiektu sklearn — ściągą

Metoda	Co robi	Na czym
.fit(X, y)	uczy estymator (model lub transformator)	zbiór treningowy
.transform(X)	przekształca dane wg nauczonych parametrów	trening i test
.fit_transform(X)	fit + transform w jednym (skrót)	tylko trening
.predict(X)	zwraca przewidywania modelu	dowolne nowe dane
.score(X, y)	domyślna miara jakości modelu	ocena modelu

Uwaga: Cały projekt housing to powtarzanie tego samego wzorca: utwórz obiekt → fit na treningu → transform/predict. Jeśli zapamiętasz parę fit/transform oraz zasadę „test tylko transform”, rozumiesz 90% mechaniki scikit-learn.